STRONG SCALING OF AUTOREGRESSIVE INFERENCE

Siavash Ahmadi *
Independent Researcher
Los Angeles, CA, 90045, USA
siaa@stanford.edu

ABSTRACT

The impressive capabilities of modern large language models critically rely on generating responses autoregressively. Autoregressive token generation properly factorizes the joint probability distribution of the output which enables superior response quality. However, following the correct statistical distribution of tokens as such destines generation to be inherently sequential, bottlenecking scaling. Efforts to overcome this have largely fallen short because they fail to preserve the output's *joint distribution*. Here, we factorize autoregressive (AR) generation into a latent variable graphical model to enable *strong* inference scaling. Our scheme reduces the number T of sequential forward passes required to generate a response with T tokens to only $O(\sqrt{T})$. In addition, it allows reducing response completion latency by increasing the number of parallel processors. We present preliminary empirical results showing generation wall clock time scales in accordance with the number of parallel processing streams. Compared to sequential generation, token throughput can be increased by up to over 70x on an entry level T4 GPU at the expense of more complexity and compute required at training. Our method's significant gains could empower better reasoning models wherein a large number of chain-of-thought tokens must be generated fast. Furthermore, because of its strong scaling nature and that it preserves AR factorization, it can be combined with established methods designed to expedite AR generation to accelerate inference even more.[1]

1 INTRODUCTION

Arguably, one of the most remarkable breakthroughs in modern AI is the discovery that the capabilities of large language models (LLMs) improve predictably as compute, model parameters, and data are scaled together (Kaplan et al., 2020; Hoffmann et al., 2022). As models continue to scale, however, the supply of high-quality training data may become a limiting factor in developing more intelligent systems. To continue improving model capabilities, inference-time scaling has emerged as a promising new frontier for maximally leveraging already trained models. Inference-time scaling allocates additional compute during generation, for example by producing intermediate

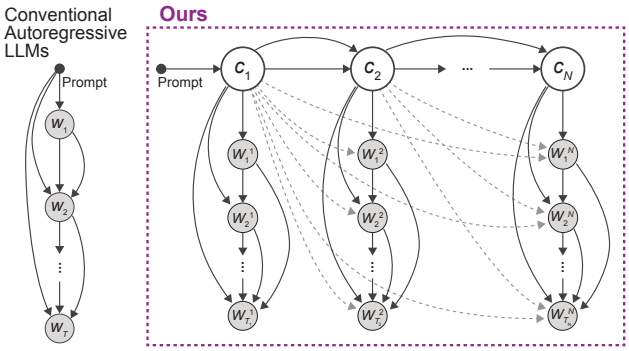

Figure 1: **Graphical model of the method**. (Left) Autoregressive token generation in modern LLMs. (Right) Our concept factorization respects the AR property on each level. Here, we have drawn the causal relations from the concepts c_i to the sentence tokens w_i with lighter dashed arrows for better figure legibility.

*Website: https://siaahmadi.github.io/

[1]This is a draft report of a work in progress. Some of the formulation and analyses are preliminary and may contain errors. Code available at https://github.com/siaahmadi/tiered_transformer.

reasoning tokens (Wei et al., 2022), generating longer reasoning traces (Wei et al., 2022; Wang et al., 2022), or exploring multiple candidate solutions (Yao et al., 2023), before attempting a task. These reasoning traces often significantly improve task performance and have enabled powerful and highly capable models (Guo et al., 2025), but also substantially increase the cost and latency of generating a single response.

In contrast to training, where distributed scaling has proven highly effective, inference faces a fundamental limitation that is rooted in model design. Specifically, LLMs model language autoregressively (AR), factorizing the joint distribution over tokens using the probability chain rule. Because this formulation preserves the exact likelihood factorization of the joint token distribution, models can be trained by directly maximizing likelihood, which empirically leads to increasingly accurate modeling of language as scale grows. However, this same autoregressive formulation imposes a fundamental constraint at inference time that even successful techniques for accelerating inference (Dao et al., 2022; Kwon et al., 2023; Pope et al., 2023; Leviathan et al., 2023) remain bound by: the sequential dependency of token probabilities. As a result, inference latency grows linearly with output length, creating a major bottleneck for serving large models and limiting the practical benefits of further inference-time scaling.

One possible approach to overcoming this limitation is to abandon the autoregressive formulation altogether, enabling more aggressive parallelization during generation. However, many non-autoregressive approaches assume conditional independence between tokens (Ghazvininejad et al., 2019; Lee et al., 2018; Stern et al., 2019), which in turn may undermine the predictable performance improvements that arise from scaling. This suggests that preserving the autoregressive formulation may be fundamentally desirable.

Here, we propose a method to decompose autoregressive factorization into strongly scalable steps. By separating thoughts from language itself (Fedorenko et al., 2024), we rewrite language modeling as a latent variable factorization (Figure 1) to autoregressively generate tokens that capture individual thoughts directly, rather than indirectly through their manifestations—the tokens. At the core of our method we replace a single decoder-only transformer with two decoder-only transformers. These decoders constitutes the two tiers of our proposed "Tiered Transformer." The top tier models the *concepts*, while the bottom tier models the language *tokens* conditioned current (latent) concepts. This segregation allows one to deploy as many parallel token decoders as there are concepts, thus greatly accelerating simultaneous text generation.

2 THE MODEL

On an abstract level, here we view language as being separate from, and a means for communicating, thought (Fedorenko et al., 2024). Thus, we treat tokens as communication building blocks that, when configured together properly, convey thoughts. We formalize this by modeling language as hierarchical generative process, positing that documents consist of a series of interrelated latent thoughts or *concepts* which give rise to specific observed token configurations.

We instantiate this modeling approach with the "Tiered Transformer," which consists of three modules: one encoder-only and two decoder-only transformers. The encoder module serves as a bidirectional map between discrete tokens and their continuous embeddings. Specifically, given a prompt $\mathcal{T} = [t_1, t_2, \ldots, t_T]$ consisting of T tokens taken from the vocabular $\mathcal{V}$, it produces a concept summary of the prompt in the embedding space ready for further processing by the decoders:

$$\text{Enc} : \mathcal{V} \to \mathbb{R}^{d_{\text{embed}}}$$

Once new token embeddings are generated by the decoders, the encoder also maps them back to discrete tokens. In between, the two decoders work together to generate new text fast.

The *concept decoder* $\text{Dec}_c : \mathbb{R}^{d_{\text{embed}}} \to \mathbb{R}^{d_{\text{embed}}}$ models latent concepts $\boldsymbol{z}$ in embedding space:

$$p(\boldsymbol{z}) = \prod_{i=1}^{m} p(z_i \mid z_1, z_2, \ldots, z_{i-1}) = \prod_{i=1}^{m} p(z_i \mid \boldsymbol{z}_{:i})$$

Algorithm 1 Tiered Transformer Inference Loop

Require: Prompts $\mathcal{T}$, concept and token count upper bounds `MAX_CONCEPTS`, `MAX_TOKENS`
Ensure: Model and tensors on CUDA device

```
Initialize output token container G
Initialize empty X with concept and token sequence length axes
i ← 0                                                    ▷ Concept counter
j ← 0                                                    ▷ Token counter
z ← Enc(T)
z_{i+1} ← Dec_c(z)
while MAX_TOKENS hasn't been reached and not both X and z empty do
    if MAX_CONCEPTS hasn't been reached then
        Concatenate [BOS] tokens to X along the concepts axis
    end if
    x_{j+1} ← Dec_t(X)                                   ▷ Token generation step
    Concatenate x_{j+1} to X along the tokens axis
    Append x_{j+1} to G                                  ▷ Token streaming
    Find [EOS]'s in x_{j+1} and drop their batches from X   ▷ Shrink towards halting condition
    j ← j + 1
    if MAX_CONCEPTS hasn't been reached then
        i ← i + 1
        Concatenate z_i to z along the concepts axis
        z_{i+1} ← Dec_c(z)                               ▷ Concept generation step
    else
        Clear z
    end if
end while
```

where $\boldsymbol{z}_{:i} = \{z_k \mid k < i\}$. Concepts can be thought of as a high-dimensional representation of an idea, summarizing a sentence or paragraph.

The *token decoder* $\text{Dec}_t : \mathbb{R}^{d_{\text{embed}}} \to \mathbb{R}^{d_{\text{embed}}}$ models tokens given a concept and existing tokens for the concept (Figure 1, right):

$$p(\boldsymbol{x}^i) = \prod_{j=1}^{n} p(x_j^i \mid z_1, \ldots, z_i, x_1^i, \ldots, x_{j-1}^i) = \prod_{j=1}^{m} p(x_j^i \mid \boldsymbol{z}_{:i+1}, \boldsymbol{x}_{:j}^i)$$

The above formulation somewhat decouples the exact identity and arrangement of tokens across sentences, paragraphs, and chapters, by implicitly capturing the intuition that far apart tokens are less likely than nearby tokens to strongly depend on each other. In addition, it enables us to strongly scale token generation at inference.

2.1 Inference

Upon the generation of the first latent embedding, the *token decoder* begins autoregressively generating new tokens, i.e., the specific configurations of tokens within a sentence or paragraph. The token decoder generates each new token conditioned on all of the previously generated concepts and all tokens generated within the current concept, thus preserving the AR property globally (Figure 1, right). Because of this partial decoupling, both decoders can run in parallel on inputs that progressively grow by concatenation with the newly generated set of tokens. Intuitively, this can be thought of as spawning independent processes that generated only a sentence or a short paragraph, but in practice the independent processes are simply treated as batches that are processed together on a CUDA device. Algorithm 1 demonstrates this process in pseudocode. The encoder and concept decoder constitute the first "tier" of the Tiered Transformer, while the token decoder constitutes its second tier.

2.2 THE ENCODER

The encoder simply converts text from the tokenizer's output to embeddings space. As in Vaswani et al. (2017), we tie the encoder's embedding layer between the input and output. In principle, the encoder can be trained as part of an end-to-end pipeline. However, here to simplify training, we will use the BERT model (Devlin et al., 2019) backbone. We apply BERT to each concept independently (see §2.5.1 for definition of a concept) and extract the `[CLS]` token as a summary of that concept. At training, this is done for each training set concept, and at inference only for the prompt (line 5 of Algorithm 1). This token has the dimensionality of the BERT backbone used, but the decoders later project it to their own dimensionality. For our experiments, we used an implementation of `bert-tiny` publicly available on HuggingFace (Bhargava et al., 2021; Devlin et al., 2019; Turc et al., 2019).

2.3 THE CONCEPT DECODER

The concept decoder is a (causal) decoder-only transformer, that learns the generative distribution of concepts. It autoregressively maps a set of continuous concept representation (which summarize the current input) to the next logical concept. We use a standard 6-layer decoder-only transformer implementation with pre-norm, sinusoidal position encodings, GeLU activation, and $p = 0.1$ dropout after both the scaled dot product attention and output projection in the attention heads. Other hyper parameters include $d_{\text{model}} = 256$, $h = 8$ attention heads, and $d_{ff} = 2048$.

2.4 THE TOKEN DECODER

The token decoder is also a (causal) decoder-only transformer with the same hyperparameters as the concept decoder, with a few differences. First, it uses two separate in-projection layers to convert the latent embeddings and tokens back to the model's dimensionality. In addition, sinusoidal positional encodings are added to all the tokens (latent concepts and new tokens). To explicitly separate the latent and token embeddings, a learned rotation matrix Q is applied to the token embedding vectors only. To ensure Q is a proper rotation matrix, we define an orthonormal projection layer as as a matrix Q constructed from a skew-symmetric matrix A, which is in turn formed from a trainable upper triangular matrix A_U as follows:

$$A_U = \begin{bmatrix} 0 & a_{12} & \cdots & a_{1d_{\text{model}}} \\ 0 & 0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & a_{(d_{\text{model}}-1)d_{\text{model}}} \\ 0 & \cdots & 0 & 0 \end{bmatrix} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

$$A = A_U - A_U^\top$$
$$Q = e^A$$

where e^A denotes the matrix exponential.

Finally, the token decoder uses the encoder's embedding layer to project the outputs back to token space. The token decoder processes batched inputs, but while the batches are logically interdependent, computationally they are completely independent. This is the source of the quadratic speedup in token throughput at inference because as the concept decoder adds to the sequence of current concepts, the token decoder simply concatenates these as batches to be processed simultaneously. As a result, the next tokens within each concept all get generated at the same time. Since this sequence grows linearly with concepts, the total number of tokens generated is a quadratic function of the number of concepts, represented as a square matrix of batched sequences.

2.5 TRAINING

2.5.1 DEFINING CONCEPT BOUNDARIES

What we consider a "concept" at training is a hyperparameter choice because exactly where the line should be drawn is unclear. One way to do this is to train on variable length concepts. Since treating the end of a sentence or paragraph as concept boundary appears to be heuristically reasonable, one could experiment with learning this from data. However, in this paper, we simplify things by treating individual sentences as concepts.

2.5.2 STAGED TRAINING

We follow a simplified training in this paper but also explore recipes for end-to-end training in the next subsection. As mentioned in §2.2, we will employ BERT as the encoder backbone of the model. BERT's weights will remain frozen during all training stages. In other word, BERT's output will provide the "ground truth" tokens for training our concept decoder and kickstart inference for new inputs.

Stage 1 (static pre-training). We train the two decoders independently. Both decoders are trained directly on BERT encodings. The concept decoder will be trained to predict BERT's `[CLS]` token for a concept given BERT's `[CLS]` token for the preceding consecutive concepts within a document. The next-token-prediction will have the following regularized regression loss:

$$\mathcal{L} = \alpha \left[\frac{1}{N} \sum_i \|\hat{c}_i - c_i\|^2 \right] + (1-\alpha) \left[1 - \frac{1}{N} \sum_i \text{cos_sim}(\hat{c}_i, c_i) \right] \tag{1}$$

where c_i is the ith concept's embedding, $\hat{c}_i$ the model's predictions, and $\text{cos_sim}(a, b) = \frac{a \cdot b}{\|a\| \cdot \|b\|}$ with $\alpha = 0.5$. This cosine similarity term is meant to encourage proper alignment of the learned vectors with those of the encoder.

The token decoder begins with a matrix of latent embeddings (representing both the prompt's and recently-generated concepts) and a `[CLS]` vector and is trained to project the a sequence of BERT's `[CLS]` embedding for consecutive concepts back to the last concept's original tokens. The token decoder is trained with the cross-entropy loss.

Stage 2 (data-informed pretraining). We counter exposure bias by employing scheduled noise sampling. Specifically, we progressively corrupt the input `[CLS]` embeddings with noise that mimics the concept decoder's prediction errors. We linearly increase the noise ratio from 0 to a target level of 0.5. The noise is sampled from a Gaussian whose standard deviation matches concept decoder's actual prediction residuals. Formally, at training step t with total steps T and noise ceiling ρ:

$$\begin{aligned} \varepsilon &\sim \mathcal{N}(0, \sigma_{M_1}^2 \cdot I) \\ \lambda(t) &= \rho \cdot \min(\frac{t}{T_{\text{warmup}}},\ 1) \\ \tilde{c}_i &= c_i + \lambda(t) \cdot \varepsilon \end{aligned}$$

where σ_{M_1} is the standard deviation of concept decoder's prediction errors. The token decoder then trains on $\tilde{c}_i$ instead of c_i.

Stage 3 (end-to-end fine-tuning). Finally, we fine-tune the decoders end-to-end with a lower learning rate.

Because at inference the decoder's are supposed to work together, in this stage we also add an auxiliary loss to encourages the concept decoder's predicted embeddings to stay close to the distribution of real BERT embeddings. We use a simple moment-matching approach to penalize

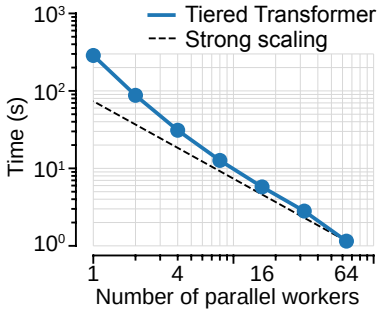

Figure 2: **(Strong) scaling of autoregressive inference.** The plot shows the time to generating 4096 tokens with the Tiered Transformer. The Y axis is shown in log scale. Strong scaling implies that by doubling the number of parallel processors the time to process the same task would halve, corresponding to a linear decrease in the log-log plane. The Tiered Transformer shows the sign of strong scaling, generating 4096 tokens from 64 concepts (64 tokens per concept) in 1.62 seconds compared to 403.3 seconds when generating 1 concept with 4096 tokens, amounting to 249x reduction in time with 64x more parallel processors. The dashed line extrapolates the expected *increased latency* by a reduction in parallel workers from 64. Note that the slight deviation from linear slope is due to two reasons. First, slow memory operations dominate in the low-parallel-worker regime but in the high-parallel-worker regime fast memory operations dominate. This is evident by comparing with the gains when doubling workers from 32 (3.96 seconds) to 64, which is closer to linear. Secondly, to simplify calculations, a factor of 2 for the number of tokens generated hasn't yet been accounted for. We are currently working to address this simplification in our code.

the difference between the mean and variance of predicted and real embeddings within each training batch:

$$\mathcal{L}_{\text{reg}} = \|\mu(\hat{c}) - \mu(c)\|^2 + \left\|\sigma^2(\hat{c}) - \sigma^2(c)\right\|^2$$

where μ and σ^2 are computed per-dimension across the batch. In stage 3, we add this to the loss in Equation 1 and replace the α hyperparameter with hyperparameters β_1, β_2, and β_3 to control the strength of the MSE loss, cosine alignment, and the latent space regularization, respectively.

2.5.3 End-to-end Training

Section to be completed.

3 Empirical results

3.1 Performance measurement method

We tried to be careful about measuring the runtime of sequential and parallel generation to have a fair comparison. Please see §5 for a discussion of the caveats. To measure time spent on the CUDA device computations, we used PyTorch's `torch.cuda.Event` object and `torch.cuda.synchronize` to synchronize the kernel. All experiments were run using a single Nvidia T4 GPU through the free tier of Google Colab.

3.2 Generation latency decreases almost linearly with additional parallel workers (Strong Scaling)

Strong scaling measures the reduction in latency to complete a fixed task with increased parallel processors. We parallelized the fixed task of generating 4,096 tokens across 1 to 64 batches (in powers of 2). Generating all tokens using a single batch represented the baseline autoregressive generation of one token for each forward pass. Dividing the same number of tokens across multiple (autoregressively interdependent) batches allowed us to measure the reduced latency, if any. In

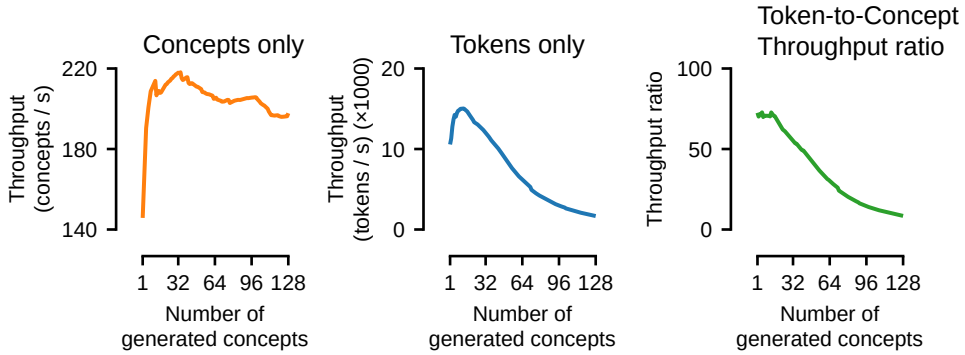

Figure 3: **(Weak) scaling of inference throughput.** Throughput is shown for concepts only (left) and tokens (middle). Concepts are generated fully autoregressively so their throughput is expected to remain constant within the GPU's capabilities. Token throughput initially rises until when there are up to about 20 concepts at which point it declines, likely due to memory management issues that arise as the processed tensor gets too big for the GPU. Together, these result in 72.8x peak throughput gains (right). The smallest fold increase occurs at 128 concepts with a 8.5x throughput acceleration. This speedup represents the fold increase in the *number of tokens* one can expect to generate with the Tiered Transformer compared to a regular decoder-only transformer using *the same number of sequential autoregressive generation steps* for both. These results were obtained suing an untrained network where the memory for concepts whose generation has completed is not dynamically de-allocated.

theory, this setup should halve the latency for each doubling of the number of batches (processed in parallel on GPU) (Figure 2, dashed line). However, inefficiencies that may arise due to the complexity introduced by the Tiered Transformer architecture, particularly slow memory operations, may lead to deviations from this expected pattern.

The Tiered Transformer autoregressively generated 4,096 tokens as a single batch in 403.3 seconds on GPU. For this task, a single latent concept embedding was first generated from a prompt which was followed by generating a first token and the subsequent tokens conditioned on the latent concept and all the previously generated tokens. We next divided the same task into generating $n_c = 2, 4, 8, 16, 32$, and 64 latent concepts autoregressively, and generating n_t tokens for each where $n_t = 4,096/n_c$. The Tiered Transformer completed these generation tasks with shorter latencies of 123.4 s, 43.7 s, 17.8 s, 8.1 s, 4.0 s, and 1.6 s, respectively, which confirm that any inefficiencies introduced by the architecture were overwhelmed by successful parallelization, achieving strong scaling (Figure 2, blue curve).

3.3 Token throughput is boosted by orders of magnitude (Weak Scaling)

We next considered the scenario where the compute budget is fixed (the number of forward passes in autoregressive generation). Here, the question is how many more tokens one could hope to generate were one to use the Tiered Transformer architecture rather than a conventional decoder-only one. As an initial step, we treated the autoregressive generation of the latent concepts in the Tiered Transformer as the baseline, since the concept decoder's operation mimics that of conventional generation almost exactly. Future work will measure this in a separate, optimized, decoder-only token generator for a stronger baseline.

The concept decoder reached a throughput of roughly 200 concepts per second across up to 128 autoregressive generation steps (alternating forward passes and memory concatenation of the newly generated latent concepts) (Figure 3, left). At the same time, the token decoder kicked in to autoregressively generating new tokens for each latent concept as soon as it became available. The token decoder reached a peak of 15,030 tokens per second at the 13th concept (Figure 3, middle), with its throughput falling to a minimum of 1,671 tokens per second at the 128th concept, still surpassing linear generation throughput. Together, the Tiered Transformer managed to increase the throughput of token generation by up to 72.8x (at 5 concepts), compared to baseline. At its slowest, the Tiered Transformer increased token throughput by 8.4x when generating tokens for 128

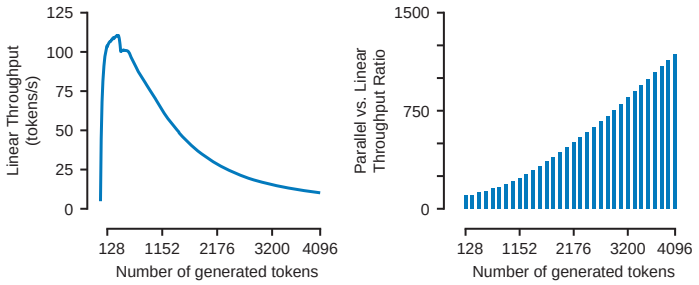

Figure 4: **Decreased time-to-tokens and increased fixed-compute throughput synergize to yield three orders of magnitude increase in throughput.** The combination of strong and weak scaling leads to orders of magnitude increase in token throughput. (Left) Token throughput with conventional autoregressive generation. (Right) The ratio of Tiered Transformer's token throughput (Figure 3, middle) to the linear generator (left plot).

concepts in parallel. Note that we measured these under the assumption that the number of concepts and tokens per concept are the same. Thus, these represent the best case throughput which is expected to drop as these parameters diverge.

We also compared the throughput of the token decoder when generating the same number of tokens in parallel versus sequentially (linear). We observed between two (102.1x, generating 128 tokens) and three (1,181.9x, generating 4,096 tokens) orders of magnitude increase in the throughput with parallel generation compared to linear generation (Figure 4).

Finally, these experiments were performed with simplifying assumptions. For example, we did not pop the batches whose generation would be considered complete (`[EOS]` token generated). While this would increase throughput by freeing up resources that can be used by other batches, managing it in memory is involved and we thus chose not to implement it yet. Note that because the concepts may emit variable-length token sequences, the throughput will be affected by the topic-sensitive empirical distribution of concept-tokens. For example, some topics might require far longer sentences (intra-concept tokens) and fewer overall concepts. In that case, the throughput would take a hit. Measuring these differences requires a trained model, which is currently in progress.

4 Related Work

Previous attempts to accelerate inference either retain the autoregressive (AR) property in generation or they violate it in order to parallelize generation. The two most successful examples of AR generation that can enjoy substantial speedups are speculative decoding and multi-token generation. In speculative decoding (Leviathan et al., 2023; Chen et al., 2023), a lightweight model—typically distilled from a larger, more capable model—generates a draft while the larger model verifies that this output does not significantly deviate from its output distribution; and if it does, the small model is directed to try again. This speeds up generation because (a) the draft is generated by a small model (hence requiring fewer FLOPs), (b) the larger model can verify the draft tokens in parallel, and (c) the small model is capable enough not to have to retry too many times. Speculative decoding has been widely adopted because it is both effective and its algorithm is based on a mathematical guarantee that it does not alter the output's distribution.

A second successful example of methods that respect the AR property, is multi-token generation (Stern et al., 2018; Cai et al., 2024). In multi-token generation several token generation heads simultaneously predict the tokens at the next k positions. The generation respects the k tokens' joint probability conditioned on the already-generated text via rejection sampling (Leviathan et al., 2023). Multi-token generation can be taken further if rather than predicting one set of k next tokens as a continuation of the current text, we generate multiple such possible continuations (Cai et al., 2024).

A fundamentally different approach admits an approximation to the tokens' exact joint distribution in exchange for significantly faster inference. The faster inference results from generating many (or possibly all) tokens at once rather than the conventional AR approach which predicts one token at a time. Such non-AR methods do reasonably well on task-specific benchmarks such as in translation

(Ghazvininejad et al., 2019; Stern et al., 2019). However, for free-form text generation such as those in popular LLM services, they almost universally suffer in quality due to failing to properly respect the interdependency of text tokens, thus exhibiting decoherence, repetitive tokens, and degraded response quality overall. Nonetheless, several non-AR approaches bear some resemblances to aspects of our method so we review them here.

Relaxing the requirement to exactly follow the correct distribution has the advantage of allowing not only gains in inference speedup, but also sequence editing capabilities. Wang et al. (2018) treat the intended output as a set of interleaved token subsequences and generate each subsequence autoregressively by conditioning on all currently available tokens to the left of the new output token. Another influential approach has been iterative refinement (Gu et al., 2017; Lee et al., 2018; Ghazvininejad et al., 2019). Lee et al. (2018) approximate the output's joint distribution by first mapping a source text into a crude draft of the target text. This draft is then refined sequentially by regenerating a new target text conditioned on the current output. The fully non-AR approach of Ghazvininejad et al. (2019) regenerates masked output tokens for a fixed number of steps to improve text quality. The Insertion Transformer (Stern et al., 2019) is another partially-AR method that predicts whether or not to insert new tokens in between the input tokens. The slots between successive input tokens get either one or zero new predicted tokens. Thus, at each forward pass the model can double the input length. As a result, it may require only $O(\log n)$ steps at inference in some translation tasks. Similarly, the Levenshtein transformer (Gu et al., 2019) casts sequence editing as a reinforcement learning task allowing insert and delete actions. These actions can be used to refine the output while accelerating generation. While these methods can be scaled to some extent and some even allow direct editing, they are only approximations to the joint output distribution. Consequently, the gains from parallel processing could be nullified by the need for additional refinement steps for longer outputs.

A final class of methods that seeks to gain the best of both worlds is diffusion-based language models (dLMs), which is a hybrid of AR and non-AR. A major advantage of dLMs is that not only do they permit output controllability, they also model the joint probability of the tokens thus enjoying high quality text generation (Nie et al., 2025). A recent example of dLM is BD3-LM (Arriola et al., 2025) which generates the output by autoregressively sampling a seed noise for the next k tokens (a "block") and uses discrete diffusion denoising to convert this noisy sample to the target text conditioned on the previously generated output. This allows the output sequence length to be flexible while allowing transformer-based LLM efficiency techniques (such as KV caching) to remain applicable. However, the denoising step is inherently sequential and may require significant compute to achieve a desirable quality, though this has been remedied by Dream (Ye et al., 2025) using ideas from consistency models (Song et al., 2023). However, it remains unclear to what extent dLMs can enjoy the benefits of scaling laws (Wu et al., 2025).

Autoregressive generation in embeddings space is somewhat rare in the literature. However, a good example that bears interesting similarities to our work is HIBERT (Zhang et al., 2019), though it is intended for a very different purpose. HIBERT is an autoencoder arranged in a hierarchical manner, and its use case is translation due to its bidirectional attention. The Tiered Transformer could also be viewed as an autoencoder though, critically, unlike regular autoencoders, it performs additional processing in the latent space before the decoding step.

5 Limitations and Future Work

Our results have several limitations that should be considered when interpreting the reported performance gains.

5.1 Production-scale validation

Our experiments were not evaluated within a fully optimized, large-scale inference stack. Modern deployments incorporate system-level optimizations such as in-flight batching, KV caching, and memory-efficient attention. The best comparison would be to evaluate our method against a production-grade autoregressive decoding pipeline that includes these techniques. Additionally, modern LLMs often rely on mixture-of-experts architectures, whose routing and communication patterns may interact differently with the parallel generation strategy proposed here. In such

settings, the overhead of spawning and coordinating additional workers could offset some of the theoretical gains.

5.2 Implementation and systems considerations

Our baseline sequential decoding implementation is intentionally simple and does not rely on compiled kernels or optimized inference runtimes. Modern inference engines often use JIT compilation or specialized kernels that can significantly accelerate sequential decoding. These optimizations may reduce the relative advantage of our approach in practice, though they could also be applied to our method.

5.3 Output quality

We did not evaluate our method's output quality at the scale of modern LLMs. A proper comparison would require training models with identical datasets, model sizes, and compute budgets across architectures, which is beyond the scope of this work. Quality degradation may also be non-uniform across tasks. For example, performance might degrade on structured reasoning tasks while remaining largely unaffected for common language generation. It is noteworthy, however, that the benefits of our approach are likely to increase with output length, making it particularly well suited for workloads involving long responses, such as reasoning-heavy models, code generation, or long-form content generation.

5.4 Interaction with other inference optimizations

Finally, we have not evaluated how the proposed method interacts with other widely used inference optimizations such as quantization. Because our current training procedure is staged rather than fully end-to-end, quantization could introduce additional degradation relative to conventional architectures. Future work should investigate whether alternative training procedures improve robustness to such optimizations.

6 Discussion

Despite enumerated limitations, the results presented here suggest that restructuring autoregressive generation to enable stronger parallelization may represent a promising direction for improving the scalability of LLM inference.

Our work can form the basis of much faster inference which can supercharge reasoning models. In addition, and more speculatively, our core idea of tiered factorization of the statistical distribution of language can be taken further by factorizing language not into two tiers but an arbitrary number of them for extreme scale inference such as advancing open-ended frontier scientific research.

Author Contributions

S.A. conceived and planned the project, implemented the code, analyzed the results, and wrote the paper. Except for project conception and planning, LLMs were consulted for brainstorming and workflow acceleration.

Acknowledgments

This project is not sponsored by any research funding sources. GPU accelerated code was run on the free version of Google Colab. Several commercial LLM services were consulted for writing parts of the manuscript. All text produced by an LLM was vetted, potentially edited, and only then accepted by the author. LLMs were also used to augment the traditional method of literature discovery (i.e., DFS on paper bibliographies). Literature so discovered was studied and when relevant, cited here by the publisher's citation reference and any appropriate URLs. Agentic code assistants were employed to accelerate code implementation. All core ideas and final decisions originated solely from the author.

REFERENCES

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Jiaqi Han, Zhihan Yang, Zhixuan Qi, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Interpolating autoregressive and discrete denoising diffusion language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=tyEyYT267x.

Prajjwal Bhargava, Aleksandr Drozd, and Anna Rogers. Generalization in nli: Ways (not) to go beyond simple heuristics, 2021. URL https://huggingface.co/prajjwal1/bert-tiny.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024. URL https://dl.acm.org/doi/10.5555/3692070.3692273.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.

Evelina Fedorenko, Steven T Piantadosi, and Edward AF Gibson. Language is primarily a tool for communication rather than thought. *Nature*, 630(8017):575–586, 2024. URL https://doi.org/10.1038/s41586-024-07522-w.

Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 6112–6121, 2019.

Jiatao Gu, James Bradbury, Caiming Xiong, Victor OK Li, and Richard Socher. Non-autoregressive neural machine translation. *arXiv preprint arXiv:1711.02281*, 2017.

Jiatao Gu, Changhan Wang, and Junbo Zhao. Levenshtein transformer. *Advances in neural information processing systems*, 32, 2019.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088. URL https://doi.org/10.48550/arXiv.2203.15556.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL https://arxiv.org/abs/2001.08361.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.

Jason Lee, Elman Mansimov, and Kyunghyun Cho. Deterministic non-autoregressive neural sequence modeling by iterative refinement. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 1173–1182, 2018.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In D. Song, M. Carbin, and T. Chen (eds.), *Proceedings of Machine Learning and Systems*, volume 5, pp. 606–624. Curan, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/file/c4be71ab8d24cdfb45e3d06dbfca2780-Paper-mlsys2023.pdf.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

Mitchell Stern, William Chan, Jamie Kiros, and Jakob Uszkoreit. Insertion transformer: Flexible sequence generation via insertion operations. In *International Conference on Machine Learning*, pp. 5976–5985. PMLR, 2019.

Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Well-read students learn better: The impact of student initialization on knowledge distillation. *CoRR*, abs/1908.08962, 2019. URL http://arxiv.org/abs/1908.08962.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Chunqi Wang, Ji Zhang, and Haiqing Chen. Semi-autoregressive neural machine translation. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 479–488, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1044. URL https://aclanthology.org/D18-1044/.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm. *arXiv preprint arXiv:2509.26328*, 2025. URL https://doi.org/10.48550/arXiv.2509.26328.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL https://hkunlp.github.io/blog/2025/dream/.

Xingxing Zhang, Furu Wei, and Ming Zhou. HIBERT: Document level pre-training of hierarchical bidirectional transformers for document summarization. In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 5059–5069, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1499. URL https://aclanthology.org/P19-1499/.